

10 Proven Games Prompts

Copy any prompt, fill the [BRACKETS], send. Mined from 500+ hours of top Claude builders.

1

Idle Clicker Game

A clicker is the ideal beginner game — simple loop, no physics — and it teaches state, persistence, and incremental progression in one self-contained file.

Build me a complete idle/clicker game that runs in a single HTML file in the browser, no build tools. Theme: [YOUR THEME, e.g. a space mining colony]. Include: a big clickable main button that earns currency, at least 5 purchasable upgrades that auto-generate currency over time with rising costs, a prestige/reset mechanic that grants a permanent multiplier, and progress saved to localStorage so it survives a refresh. Make the numbers feel satisfying with clean formatting (1.2K, 3.4M). Stay in plan mode and ask me clarifying questions first.

2

2D Platformer

Platformers test movement feel and level design; asking for 'arcade-feel' and 'real weight' up front avoids the common problem of unplayable, floaty controls.

Build me a 2D side-scrolling platformer that runs in the browser on an HTML canvas. Requirements: arrow-key / WASD movement with jumping that has real weight (gravity, acceleration, a satisfying jump arc), at least 3 hand-designed levels with platforms, collectible coins, and one hazard type. Add a health system, a level-complete screen, and a simple parallax background. Make controls responsive and arcade-feel — easy to pick up. Plan it first and break the build into atomic tasks.

3

Match-3 Puzzle Game

Match-3 forces real grid logic — swap validation, cascade resolution, gravity refill — making it a strong, distinct puzzle build.

Build me a match-3 puzzle game (Bejeweled / Candy Crush style) as a single-page web app. Requirements: an 8x8 grid of 6 colored gem types, click-or-drag to swap adjacent gems, automatic detection and clearing of matches of 3+, gems falling to fill gaps with new ones spawning on top, a combo/cascade multiplier, a score counter, and a 60-second timed mode. Add smooth swap and fall animations. Build and test it section by section, and verify the match logic actually works with a screenshot.

4

Endless Runner

Endless runners are one-input, infinitely replayable, and teach procedural generation and difficulty pacing — a genuinely different genre from the others.

Build me an endless runner game for the browser (Chrome-dinosaur / Flappy-Bird energy). Theme: [YOUR THEME]. The player auto-runs and the world scrolls; one input (spacebar / tap) jumps or flaps. Procedurally spawn obstacles at increasing speed, track distance and a high score saved to localStorage, and show a clean game-over screen with a restart button. Tune the difficulty curve so the first 10 seconds are forgiving and it ramps up smoothly.

5

Tower Defense Game

Tower defense combines pathfinding, an economy, and wave management — a strategy genre that exercises more systems thinking than arcade games.

Build me a tower defense game that runs in the browser. Requirements: a fixed path enemies walk along, 3 distinct tower types (e.g. rapid-fire, splash, slow) the player places on buildable tiles for currency, waves of enemies with increasing health, currency earned per kill, a lives counter that drops when an enemy reaches the end, and an upgrade option per tower. Include a 'start wave' button and a win/lose state. Plan the architecture first — keep enemies, towers, and the wave manager as separate modules.

6

Snake (Arcade Classic)

Snake is the perfect small, fully-playable first game — tight scope, instantly testable, and great for learning the build-then-iterate loop.

Build me a polished version of the classic Snake game on an HTML canvas. Beyond the basics, add: smooth grid-based movement, food that increases length and score, a gradually increasing speed, wrap-around-vs-wall-collision as a toggleable setting, a pause key, a high score saved to localStorage, and a subtle modern visual style (not default green-on-black). Make it feel crisp and responsive. One-shot the foundation, then we'll iterate.

7

Two-Player Local Multiplayer

Local multiplayer introduces dual input handling and competitive scoring without needing any networking — a distinct, high-fun build.

Build me a local two-player game playable on one keyboard in the browser — pick [Pong / air hockey / a tank-duel arena]. Player 1 uses WASD, Player 2 uses the arrow keys. Requirements: smooth physics for the ball/puck/projectiles, a first-to-7 score system, a center-line and clean court visuals, a countdown before each round, and a match-winner screen with a rematch button. Make collisions feel punchy. Plan it first and ask me which game I want.

8

Text-Based Adventure / RPG

A text adventure is logic-and-state heavy with almost no graphics — a completely different build from the arcade games and great for non-visual creativity.

Build me a browser-based text adventure RPG as a single page. Theme: [YOUR THEME, e.g. a haunted mansion]. Requirements: room-to-room navigation via on-screen choice buttons, an inventory system, at least 3 simple puzzles that gate progress (find an item to unlock a path), a basic turn-based combat encounter with HP, a save/load via localStorage, and at least two possible endings. Keep the writing atmospheric and concise. Outline the full story map before you write any code.

9

Memory / Card-Matching Game

A memory game is a clean, satisfying puzzle build that teaches game state, timers, and animation in a low-stakes package distinct from action games.

Build me a memory card-matching game for the browser. Requirements: a grid of face-down cards (let me choose 4x4 or 6x6), click to flip, match pairs by icon/emoji, cards stay revealed when matched and flip back when they don't, a move counter and a timer, a win screen showing moves and time, and a difficulty selector. Add smooth card-flip animations and a clean modern look. Verify the matching logic with a screenshot test.

10

Top-Down Arena Shooter

An arena shooter exercises real-time movement, aiming, spawning, and collision at scale — the most action-heavy genre in the set and a satisfying capstone build.

Build me a top-down wave-survival arena shooter for the browser on an HTML canvas. Requirements: WASD movement, mouse-aim shooting, enemies that spawn in escalating waves and chase the player, a health bar, score and wave counters, at least one power-up drop (e.g. faster fire rate), and a game-over screen with restart. Keep performance smooth with many enemies on screen. Make takeoff easy: forgiving early waves, then ramp difficulty. Build and playtest it in a loop until it feels good.

11

Rhythm / Beat Game

A rhythm game is built entirely on precise timing windows and input latency — a fundamentally different challenge from physics or grid games, and very satisfying when tuned.

Build me a rhythm game that runs in a single HTML file in the browser. Requirements: 4 vertical lanes with notes that fall toward a hit-line at the bottom, key presses (D, F, J, K) to hit notes as they cross the line, timing windows that grade each hit as Perfect / Good / Miss, a combo counter, a score that rewards accuracy and combos, and a results screen with accuracy percentage. Let me define a simple note chart as an array of {lane, time} so I can author my own song. Make the hit feedback feel crisp and punchy.

12

Physics Sandbox

A physics sandbox has no scoring or win state — it's pure simulation, teaching collision response and stable physics integration, an angle none of the other games cover.

Build me a 2D physics sandbox that runs in the browser on an HTML canvas. Requirements: gravity, and the ability to click-and-drag to spawn objects (balls, boxes, planks) that collide and stack realistically, plus mouse-drag to fling objects. Add a toolbar to switch object types, a gravity slider, a 'pause/play simulation' button, and a clear-all button. No win condition — it's a toy. Keep the physics stable so stacked objects don't jitter or explode. Plan the collision model first.

13

Card Game (Solitaire / Blackjack)

Card games demand correct rule enforcement, a shuffled deck model, and turn logic — a distinct logic-heavy build that's instantly recognizable and fun to ship.

Build me a complete browser card game in a single HTML file — pick [Klondike Solitaire / Blackjack]. Requirements: a properly shuffled 52-card deck, clean card visuals with suits and ranks, full rules enforced (legal moves only), drag-and-drop or click-to-play interaction, a score or chip count, a win/lose state, and a 'new game' button. For solitaire add auto-complete when the board is solvable; for blackjack add a dealer AI that follows standard hit-on-16 rules. Plan the card and deck data model before building.

14

Maze Game With Procedural Generation

A maze game centers on procedural generation with a guaranteed-solvable constraint and grid traversal — a clean algorithmic build distinct from the action and puzzle genres.

Build me a maze game for the browser on an HTML canvas. Requirements: a procedurally generated maze that is always solvable (use a real maze algorithm like recursive backtracking), arrow-key / WASD movement of a player from start to exit, a timer, a level counter that generates a new larger maze each time you finish, and an optional 'show solution path' button. Add a minimalist top-down look. Generate a fresh maze every level so it's endlessly replayable. Plan the generation algorithm first.

15

Typing Speed Game

A typing game turns the keyboard itself into the controller and teaches live input handling, WPM math, and difficulty scaling — a genuinely different and replayable genre.

Build me a typing game that runs in a single HTML file in the browser. Theme: [YOUR THEME, e.g. words falling like meteors that you defuse by typing them]. Requirements: words appear and the player types them to clear them, words appear faster as the game goes on, a lives system (a missed word costs a life), live words-per-minute and accuracy stats, a score, and a game-over screen showing final WPM and accuracy. Pull words from a built-in list I can edit. Make the visual feedback satisfying when a word is cleared.

